



← Back to student guides

 ClearML

Experiment tracking

3 levels

116 sections



The Complete Guide

Taught at three levels, written three ways. Pick Beginner, Mid-level, or Senior, then read the full explanation, a fast interview review, or the practical tips and traps for that level.

TASK

QUEUE

AGENT

MODEL

17

sections

23

examples





work you own when ClearML is your team's platform rather than your tool.

Up to this point the questions have been about making a run reproduce. From here they are different: who can enqueue code onto your GPU fleet, whether a model from eighteen months ago can still be rebuilt and proven, what the platform costs and who pays for it, which component breaks first when the team triples, what happens when Elasticsearch fills up on a Friday night, and where ClearML stops being the right tool.

Where this picks up

Topic from earlier levels	What this level adds
The hosted server	Self-hosted deployment: three data stores, storage, TLS, and who owns upgrades
Credentials	The trust model : enqueue is code execution; scoped keys, service accounts, rotation
Projects and tags	Multi-tenancy, RBAC, quota, and isolation between teams
<code>output_uri</code>	A real storage architecture: buckets, lifecycle, egress, and retention policy
Agent caches	Fleet economics: autoscalers, spot instances, GPU utilisation as a metric
Scalars and Elasticsearch	Capacity planning, index growth, and what breaks first
<code>dvc</code> -free reproducibility	Environment pinning to the image digest, and rebuild verification
The registry	Lineage and audit that satisfies a reviewer or a regulator
Datasets	Erase requests against an immutable, content-addressed store



CI	Fork trust, ephemeral runners, and enqueue as a privileged operation
Debugging	Incident playbooks: lost server, corrupted index, unreproducible model
new	Backup and restore · upgrade procedure · platform ownership · where ClearML ends

I am starting with the trust model, because every other decision in this track depends on it.

The trust model: enqueue is code execution

The single most important sentence about ClearML security: **enqueueing a task is arbitrary code execution on your agents**. A task carries a repository, a commit, a diff, a package list, and an entry point. An agent claims it and runs it. Anyone who can enqueue can run code on your GPU fleet, with whatever credentials that fleet holds.

That reframes the whole access question around "who can execute" instead of "who can see the experiments".

WHO CAN ENQUEUE. THIS IS THE EXECUTION SURFACE

Researcher

Enqueue to **cpu** / **gpu** . Never to serving or production queues

CI on a protected branch

Enqueue to **ci** only. No production cloud credentials

CI on a fork PR

Nothing, or an isolated ephemeral queue holding no other access



Platform team + schedulers · registry read/write

cpu · **gpu**

Researchers · data read, artifact write

ci-untrusted

Ephemeral hosts · **no credentials at all**



Keys are workspace-wide

One leaked researcher pair reads every experiment, dataset pointer, and model, and can enqueue

The diff is on the server

An untracked **.env** in your tree lands in the task record, and in your backups

Agents hold ambient creds

Any task landing on a worker inherits that worker's cloud access

The three rows at the bottom compose into one risk: a workspace-wide key plus an agent with production credentials means whoever holds that key can run code with that access. Queue-level isolation is therefore a security control rather than tidiness, and fork pull requests are the case where it matters most.

Actor	Needs	Must not have
A researcher	Read all; write own tasks; enqueue to cpu / gpu	Enqueue to production or serving queues
CI on a protected branch	Create and enqueue to ci ; read models	Enqueue to gpu with production credentials
CI on a fork pull request	Nothing , or an isolated ephemeral queue	Any workspace credential



An agent host	its own worker credentials, read the repo; write artifacts	Delete on the artifact bucket
A serving service	Read the registry; read model artifacts	Write anything
A retention job	Delete, scoped and audited	Write or enqueue
A vendor or contractor	Read on one project	Enqueue anywhere

Three properties of ClearML's model make this sharper than it first looks:

An access/secret pair is workspace-wide by default. It is not scoped to a project. A leaked researcher key reads every experiment, every dataset pointer, and every model in the workspace, and can enqueue.

The uncommitted diff is stored on the server. That is the feature that makes a dirty working tree reproducible. It also means anything sitting in your working tree at run time (a `.env` you forgot to gitignore, a notebook with a pasted token) is now in the task record, readable by everyone with workspace access, and captured in your backups.

Agents hold ambient credentials. A GPU worker typically has cloud credentials so it can read datasets and write artifacts. Any task that lands on it inherits them. That is why fork pull requests are dangerous and why queue-level isolation is a security control, not tidiness.

The most dangerous configuration in ClearML

A public repository with a `pull_request` workflow that holds workspace credentials and enqueues to a queue whose agents have production cloud credentials. Any stranger who opens a pull request gets code execution with those credentials. Fork builds must run on ephemeral agents in an isolated queue with no other access, or be gated behind explicit human approval before they can execute.

The access shape that works:

queues as a security boundary

Copy



```
cpu          ← enqueue: researchers.  
ci           ← enqueue: CI on protected branches.  
ci-untrusted ← enqueue: fork PRs after approval.  
serving      ← enqueue: platform team only.
```

```
creds: data read, artifact write  
Creds: artifact write only  
Creds: NONE. Ephemeral hosts  
Creds: registry read
```

► TRY IT

1. List every place a ClearML access/secret pair exists: laptops, CI secrets, agent hosts, notebooks, docs. Count them.
2. For each one, write down what it can do. Note how many are workspace-wide when they only needed one project.
3. Open three recent tasks and read Execution → Uncommitted Changes. Look for anything that should not be there.
4. Check whether a fork pull request in any public repository can reach a queue whose agents hold cloud credentials.

Expected result – more credentials than expected, all broader than needed, and, in step three, at least one diff containing something you would rather not have stored. Step four is the one to fix today if the answer is yes.

Credentials, service accounts, and rotation

At Mid the goal was "not committed". At this level the goal is **short-lived, scoped, and attributable**.

BASH

Copy

```
# Per-purpose credentials, never shared. Each is separately revocable.  
# Settings → Workspace → App Credentials, one per:  
#   - each CI pipeline  
#   - each agent pool  
#   - each serving service  
#   - each human (personal, never in automation)
```



```
# systemd unit or Kubernetes secret – nothing on disk in a home directory
Environment=CLEARML_API_HOST=https://clearml.internal/api
Environment=CLEARML_WEB_HOST=https://clearml.internal
Environment=CLEARML_FILES_HOST=https://clearml.internal/files
Environment=CLEARML_API_ACCESS_KEY=...      # from the secret store
Environment=CLEARML_API_SECRET_KEY=...
Environment=CLEARML_WORKER_ID=gpu-a100-01   # attributable in Workers & Queues
```

For the cloud credentials the agent needs, the goal is that there is nothing to store at all:

✓ Workload identity

- The agent's node identity grants bucket access
- IRSA on EKS, workload identity on GKE, instance profiles on EC2
- Nothing on disk, nothing to rotate, nothing to leak
- Scoped per node pool, so a queue's access is a cluster policy

✗ Long-lived keys in `clearml.conf`

- A file on every worker, readable by every task on it
- Rotation means touching every host
- Any task that lands there inherits full access
- Present in backups and in machine images

Control	Why it matters
One credential per purpose	Revocation is surgical, and the audit log names a caller
<code>CLEARML_WORKER_ID</code> set explicitly	A misbehaving worker is identifiable rather than "worker-3"
Workload identity for cloud access	The agent's bucket access is a cluster policy, not a file
Rotation on a schedule, with a drill	An unexercised rotation procedure does not work when you need it
Secrets in a real store	Vault, Secrets Manager, or sealed secrets: never a repo or a wiki
<code>.env</code> files gitignored, and verified	Otherwise they land in the stored diff of every run



a service-account JSON, or a notebook with a pasted token that is present but untracked ends up in the task record, visible to the whole workspace and persisted in backups. Add a pre-run check, gitignore aggressively, and if it happens, remember that deleting the task is the only remediation and the credential must still be rotated.

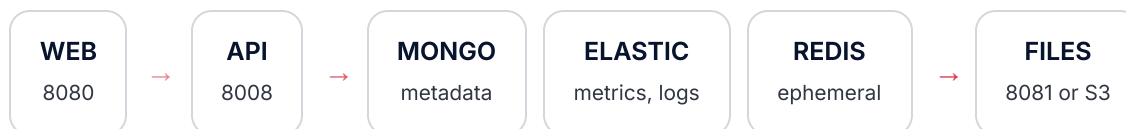
► TRY IT

1. Create a purpose-scoped credential for one CI pipeline and switch that pipeline to it. Confirm the old shared key is now unused.
2. Revoke a credential and watch what fails. That is your blast radius.
3. Move one agent pool to workload identity and confirm it can still read data and write artifacts with no keys on disk.
4. Deliberately leave an untracked `secrets.txt` in a working tree, run a task, and find it in the stored diff.

Expected result – step four is the demonstration that changes behaviour. Seeing your own file contents in a task's Uncommitted Changes tab is more persuasive than any policy document, and it takes thirty seconds.

Self-hosting: what you own

The hosted server hides five components. Self-hosting means owning all of them, plus their failure modes.



Component	Holds	If it dies	Growth driver
MongoDB	Tasks, projects, models, users, queues	Everything stops; this is the system of record	Task count



Elasticsearch	Scalars, plots, console logs	metrics and logs unavailable, tasks still run	Reporting rate x runs
Redis	Locks, ephemeral worker state	Workers misbehave; usually self-healing	Fixed, small
API server	Stateless	Stateless: restart or scale out	-
Web server	Stateless	UI down; agents keep running	-
File server	Artifacts, models, debug images	Artifacts unreadable; replace it with S3	Artifact volume

docker-compose.yml: the parts that matter

Copy

```
services:
  elasticsearch:
    image: elasticsearch:8.x
    environment:
      ES_JAVA_OPTS: "-Xms4g -Xmx4g"          # ≤ 50% of container RAM, and never > 31g
      bootstrap.memory_lock: "true"
      cluster.routing.allocation.disk.watermark.low: "90%"
      cluster.routing.allocation.disk.watermark.high: "95%"
    volumes: ['/opt/clearml/data/elastic_7:/usr/share/elasticsearch/data']

  mongo:
    image: mongo:6.x
    volumes: ['/opt/clearml/data/mongo_4/db:/data/db']

  apiserver:
    image: allegroai/clearml:latest
    environment:
      CLEARML__services__async_urls_delete__enabled: "true"  # actually free storage
      depends_on: [mongo, elasticsearch, redis]
```

Elasticsearch is what breaks first, and disk is why

Metrics and console logs grow with every run and never shrink on their own. When the high disk watermark is crossed, Elasticsearch marks indices read-only, and the symptom is that **experiments stop reporting metrics while training continues**, a silent and



BASH

Copy

```
# The three health checks worth having on a dashboard
curl -s localhost:8008/debug.ping
curl -s 'localhost:9200/_cluster/health?pretty' | grep -E 'status|unassigned'
curl -s 'localhost:9200/_cat/indices?v&h=index,docs.count,store.size&s=store.size:desc'

# Release a read-only block after clearing disk space
curl -X PUT 'localhost:9200/_all/_settings' -H 'Content-Type: application/json' \
  -d '{"index.blocks.read_only_allow_delete": null}'
```

Non-negotiables for a server anyone else depends on:

Requirement	Why
TLS terminated at a reverse proxy	Credentials and payloads cross the network in the clear otherwise
Artifacts in S3, not the file server	The file server is a single disk with no lifecycle or replication
Separate volumes for Mongo and Elastic	One filling up must not take the other down
Backups of Mongo and Elastic, tested	Mongo alone restores tasks with no metrics or logs
Pinned server image tag	latest means an unplanned migration on the next restart
Disk, memory, and cluster-health alerts	Elasticsearch read-only is silent from the user's side

► TRY IT

1. On a test instance, list the Elasticsearch indices by size and identify what is growing.
2. Fill the Elasticsearch volume past the high watermark on purpose and observe what a running task does. Then release the block.
3. Confirm your artifacts go to S3 rather than the file server, by reading an artifact's URL.



the failure you must recognise instantly. Step four is a thirty-second check that prevents an unplanned migration during someone else's deadline.

Backup, restore, and upgrades

You have backups. Whether you have ever restored one is the open question.

A ClearML backup has **three** parts, and taking only the first is the common mistake:

01 MongoDB

`mongodump` of the `backend` and `auth` databases. This is the system of record: tasks, projects, models, users, queues.

02 Elasticsearch

A snapshot to a repository. Without it a restore gives you tasks with no scalars, no plots, and no console logs.

03 Artifact storage

Your bucket, with versioning enabled. If you still use the file server, its volume.

BASH

Copy

```
# 1. Mongo
docker exec clearml-mongo mongodump --archive=/tmp/mongo.gz --gzip \
  --db backend --db auth
docker cp clearml-mongo:/tmp/mongo.gz ./backup/

# 2. Elasticsearch – register a repository once, then snapshot
curl -X PUT 'localhost:9200/_snapshot/backups' -H 'Content-Type: application/json' -d '{
  "type": "fs", "settings": {"location": "/usr/share/elasticsearch/snapshots"}}'
curl -X PUT "localhost:9200/_snapshot/backups/snap-$(date +%F)?wait_for_completion=true"
```



The restore drill is the part that matters, and it must be rehearsed on a separate host:

BASH

Copy

```
docker compose down
docker exec -i clearml-mongo mongorestore --archive --gzip < ./backup/mongo.gz
curl -X POST "localhost:9200/_snapshot/backups/snap-2026-08-01/_restore"
docker compose up -d
# Then verify, in this order:
# 1. Can you log in?
# 2. Does a known task still show its scalars? ← proves Elastic restored
# 3. Can you download a known artifact? ← proves storage config
# 4. Can an agent claim a queued task? ← proves the fleet reconnects
```

Upgrades follow the same discipline, because a server upgrade runs data migrations:

Step	Detail
Read the release notes for every version you skip	Migrations are cumulative and some are one-way
Back up all three stores first	An interrupted migration is why you have backups
Pin the target tag, never latest	So the change is deliberate and repeatable
Upgrade the server before the agents	New agents may require newer API endpoints
Keep SDK and agent versions paired	An SDK ahead of the server can send fields it rejects
Announce a window	Agents survive a short API outage; running tasks buffer, but not forever

Elasticsearch snapshots are the part people skip, and metrics are unrecoverable without them

A Mongo-only backup restores a complete-looking system: every task, every model, every parameter, and empty Scalars tabs, because scalars, plots, and console output live



► TRY IT

1. Take all three backups today. Note how long each takes and how large it is.
2. Restore them onto a scratch host and run the four verification steps in order.
3. Specifically confirm that a task's scalars came back. If they did not, your Elasticsearch snapshot is missing or broken.
4. Write down your measured RTO, from "the server is gone" to "an agent claimed a task", and tell your team the number.

Expected result – a real recovery time instead of an assumed one, and often a missing Elasticsearch snapshot. Step four is what turns "we have backups" into a commitment somebody can plan around.

Multi-tenancy, RBAC, and quota

One workspace shared by four teams degrades predictably: a flat project list nobody can search, a GPU queue one team monopolises, and no way to tell who spent the storage.

The open-source server has a **single workspace and no role-based access control**: every user with credentials can see and do everything. That is a real limitation and you should state it plainly rather than discover it during a security review. RBAC, multiple workspaces, and per-user permissions are paid-tier features.

What you can do with the open-source server:

Control	Mechanism	Limit
Structure	Nested projects: <code>team/subteam/purpose</code>	Convention only; nothing enforces it
Scheduling fairness	Separate queues per team, with fixed worker counts	Workers are the quota



Execution isolation	Separate agent pools with separate cloud credentials	The strongest control available
Attribution	Mandatory tags for team and cost centre, set in code	Convention, but auditable
Storage isolation	A bucket prefix per team, with its own lifecycle rules	Enforced by cloud IAM, not by ClearML
Hard isolation	A separate ClearML server per tenant	Real cost, real operational duplication

a workable convention for one shared server

Copy

```
projects:
  platform/...      ← pipelines, schedulers, serving control
  team-vision/...   ← their experiments
  team-nlp/...
  datasets/...      ← shared, write-restricted by process
queues:
  vision-gpu (2 workers) · nlp-gpu (2 workers) · shared-gpu (2, spot) · services (1)
storage:
  s3://ml-artifacts/team-vision/...   lifecycle: 90d → IA, 365d → delete
  s3://ml-artifacts/team-nlp/...
  s3://ml-datasets/...                versioning on, write-restricted
```

enforce attribution at the SDK level

Copy

```
# In a shared internal wrapper every team imports:
def init_task(project: str, name: str, **kwargs):
    team = os.environ["ML_TEAM"]                # from the environment
    task = Task.init(project_name=f"{team}/{project}", task_name=name,
                      output_uri=f"s3://ml-artifacts/{team}/clearml", **kwargs)
    task.add_tags([f"team:{team}", f"cost-centre:{os.environ['ML_COST_CENTRE']}"])
    return task
```

A thin internal wrapper is the highest-leverage governance tool you have



than documentation, because nobody has to remember anything, and it gives you one place to change when the storage layout or the tag vocabulary changes.

► TRY IT

1. Sort your workspace's projects by task count and see whether the structure reflects teams or accidents.
2. Check whether one team can currently occupy every GPU worker. If so, split the queues.
3. Try to answer "how much storage did team X use last month" from the data you have today.
4. Write the wrapper above and switch one team's scripts to it.

Expected result – step three is usually impossible without prefixes and tags, which is exactly the point: cost attribution has to be designed in before the bill arrives, because it cannot be reconstructed retroactively.

Cost: what the platform costs

Four cost centres, and only one of them is usually noticed.

Centre	Driver	Control
GPU-hours	Trials × epochs × idle time	HPO budgets, autoscaling, session timeouts, utilisation monitoring
Artifact storage	Checkpoints per run × runs × retention	Keep best+last, lifecycle rules, <code>save_top_k_tasks_only</code>
Dataset storage	Versions × copies	<code>add_external_files</code> , squashing, deduplication
Server infrastructure	Task count and reporting rate → Mongo and Elastic disk	Retention policy, reporting discipline



Egress	Cross-region and cross-cloud data movement	Co-locate compute and storage
--------	--	-------------------------------

Idle GPU time is nearly always the largest single line, and it is invisible unless you measure it:

a weekly utilisation report, itself a scheduled task

Copy

```
from datetime import datetime, timedelta
from clearml import Task

week = datetime.utcnow() - timedelta(days=7)
tasks = Task.get_tasks(task_filter={"status": ["completed", "failed", "aborted"]})

gpu_seconds, wasted = 0.0, 0.0
for t in tasks:
    if t.data.last_update < week or "gpu" not in (t.data.execution.queue or ""):
        continue
    runtime = (t.data.completed - t.data.started).total_seconds()
    gpu_seconds += runtime
    if t.status in ("failed", "aborted"):
        wasted += runtime

print(f"GPU hours: {gpu_seconds/3600:.1f}  wasted on failed/aborted: {wasted/3600:.1f}")
```

Waste source	Typical share	Fix
Abandoned interactive sessions	Often the largest	Session timeout; a report naming the owner
Failed runs that fail late	Large and demoralising	Fail fast: validate config and data access in the first minute
Sweeps with no pruning	Silent and total	Verify the objective; <code>min / max_iteration_per_job</code>
Checkpoints every epoch	Storage, not compute	Best + last only
Workers idle between bursts	Real money on cloud GPUs	Autoscaler with a scale-to-zero floor



```
from clearml.automation.auto_scaler import AutoScaler, CloudDriver

AutoScaler(
    hyper_params=AutoScaler.Settings(
        max_idle_time_min=10,          # terminate an idle instance after 10 minutes
        polling_interval_time_min=2,
        max_spin_up_time_min=25,
        workers_prefix="dyn",
    ),
    driver=CloudDriver(...),          # AWS/GCP driver with instance types per queue
).start()
```

Spot instances need `retry_on_failure` , or you have bought cheaper failures

Preemption terminates a task mid-run. Without retries at the pipeline level and periodic checkpointing plus `continue_last_task` in the training code, a 70%-cheaper instance that dies at epoch 90 of 100 costs more than an on-demand one. Spot is only cheap for jobs that can resume.

► TRY IT

1. Run the utilisation report and compare GPU-hours consumed against hours that produced a completed run.
2. List every running `clearml-session` and how long each has been up. Name the owners.
3. Total your artifact bucket and work out what fraction is sweep trials that were never promoted.
4. Price one week of your GPU fleet at on-demand, spot with retries, and autoscaled-to-zero.

Expected result – the failed-and-abandoned share is usually shocking the first time anyone measures it. Step two is the fastest win available: idle sessions are pure waste and the fix is a timeout plus a weekly report naming names.

Scale: which component breaks first



1 Reporting rate

Per-batch scalars from many concurrent runs. Elasticsearch write pressure; scalars lag, then indices go read-only.

2 Task count

Hundreds of thousands of tasks. MongoDB queries slow; the experiments table and search get sluggish.

3 Artifact volume

Storage cost, and the file server's single disk if you never moved to S3.

4 Worker count

Hundreds of agents long-polling. API server CPU; scale it horizontally, it is stateless.

Dimension	First symptom	Fix in order
Reporting rate	Scalars minutes behind	Report per epoch not per batch → more Elastic RAM/disk → separate the cluster
Task count	Slow table and search	Archive and delete old CI/sweep tasks → Mongo resources → retention policy
Artifact volume	Bill, or a full file-server disk	<code>output_uri</code> to S3 → lifecycle rules → keep best+last only
Worker count	API latency, claim delays	Scale API replicas → tune polling interval → shard by server if huge
Console log volume	Elastic disk	Do not print in a tight loop; log at a sane level

Reporting discipline is the cheapest and most effective lever, and it is a code change rather than an infrastructure one:



```
logger.report_scalar(loss, "train", loss, step)

# Good: per epoch, plus a moving average within the epoch
for epoch in range(epochs):
    epoch_loss = train_one_epoch()
    logger.report_scalar("loss", "train", epoch_loss, epoch)

# When you genuinely need intra-epoch resolution, sample it
if step % 100 == 0:
    logger.report_scalar("loss", "train_batches", loss, step)
```

Retention is what keeps Mongo and Elastic bounded. It has to be a scheduled task, because nobody does it by hand:

ops/retention.py: a services task on a schedule

Copy

```
from datetime import datetime, timedelta
from clearml import Task

RULES = [
    ("vision/ci", 30, ("failed", "aborted", "completed")),
    ("vision/hpo", 60, ("failed", "aborted")),
    ("vision/experiments", 365, ("failed", "aborted")),
]

for project, days, statuses in RULES:
    cutoff = datetime.utcnow() - timedelta(days=days)
    for t in Task.get_tasks(project_name=project):
        if t.data.last_update ≥ cutoff or t.status not in statuses:
            continue
        if set(t.get_tags()) & {"keep", "published", "paper"}:
            continue # an explicit exemption, always needed
        print(f"delete {t.id} {t.name}")
        # t.delete(delete_artifacts_and_models=True) ← enable after a dry run
```

Never let a retention job delete anything a model still needs

Deleting a training task can remove the artifacts and weights a registered, possibly production, model points at. Exempt any task that produced a published model, anything



► TRY IT

1. Find your highest-reporting task and count its scalar points. Compare it against a per-epoch equivalent.
2. Count tasks by project and identify what is filling the system. It is usually CI and sweeps.
3. Run the retention script in report-only mode and read the list, plus the total artifact size it would free.
4. Check the exemption logic against a task that produced a published model.

Expected result – the count in step one is usually two orders of magnitude larger than anyone expects. Step four is the one that prevents an incident: a retention job that deletes a production model's weights is a self-inflicted outage with no undo.

Reproducibility that survives eighteen months

ClearML records the commit, the diff, and the Python packages. That is necessary and not sufficient. Everything below the Python layer is unpinned unless you pin it.

Layer	Recorded by default	How to pin it
Application code	Yes: commit + diff	Nothing more needed
Python packages	Yes, but as detected names/versions	A hash-pinned lock file, committed
Python version	Yes, recorded	The base image
CUDA, cuDNN, drivers	No	The base image digest
System libraries	No	The base image digest



Data	Only if you used a Dataset	Dataset version, consumed with <code>alias</code>
Random seeds	Only if you connected them	A connected parameter
Non-determinism in the framework	No	<code>torch.use_deterministic_algorithms(True)</code> , or accept and record it

the pinning pass, in full

Copy

```
task.set_base_docker(
    # A digest, not a tag. Tags move; digests do not.
    "myregistry/train@sha256:9f2c4b19e0a7d3f1c6b8a2e4d7091f3b8a2e4d7091f3b8a2e4d7091f3b8
    docker_arguments="--shm-size=8g",
)
task.set_packages("requirements.lock") # hash-pinned, committed

env = task.connect({"seed": 42, "deterministic": True}, name="reproducibility")
torch.manual_seed(env["seed"]); np.random.seed(env["seed"]); random.seed(env["seed"])
if env["deterministic"]:
    torch.use_deterministic_algorithms(True)
    torch.backends.cudnn.benchmark = False

data = Dataset.get(dataset_project="datasets", dataset_name="iris",
                    dataset_version=params["dataset_version"],
                    alias="training data").get_local_copy()
```

The part that makes it a claim rather than a hope is a scheduled rebuild of a past release:

ops/rebuild_check.py: weekly, on the services queue

Copy

```
from clearml import Task

RELEASE_TASK_ID = "...the task behind the current production model..."
TOLERANCE = 0.002

original = Task.get_task(task_id=RELEASE_TASK_ID)
clone = Task.clone(source_task=original, name=f"rebuild check {original.name}")
Task.enqueue(clone, queue_name="gpu")
clone.wait_for_status(status=("completed", "failed", "aborted"))
```



```
delta = abs(acc(clone) - acc(original))
Task.current_task().get_logger().report_single_value("rebuild_delta", delta)
assert clone.status == "completed", f"rebuild failed: {clone.get_output_log_web_page()}"
assert delta ≤ TOLERANCE, f"rebuild drifted by {delta:.4f}"
```

"We are reproducible" is a claim until something checks it weekly

Every unpinned layer degrades silently: a base image tag gets rebuilt, a package yanked, a CUDA minor bump changes a kernel. A scheduled rebuild of a past release is the only mechanism that converts reproducibility from an assumption into a monitored property, and the first time it runs, it will fail. That failure is the point.

► TRY IT

1. Take the task behind your current production model. Clone it and run it on an agent today.
2. Compare the metric. Write down the delta.
3. For every layer in the table above, note whether that task pins it. Count the gaps.
4. Fix the base image to a digest and add the lock file, then repeat.

Expected result – almost always a failure or a drift on the first attempt, and a short list of unpinned layers. Step three is the deliverable: a named list of gaps is far more useful to your team than a vague sense that reproducibility could be better.

Lineage, provenance, and audit

For regulated work, or any post-incident review, you need to answer four questions about a deployed model without a conversation:

01 What code produced it?



02 What data produced it?

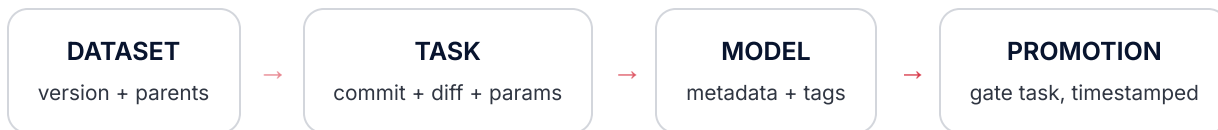
The task's configuration records the dataset id via `alias` ; the dataset records its parents.

03 What configuration produced it?

The task's hyperparameters and configuration objects, as executed.

04 Who approved it and when?

The promotion task: who ran it, what it compared, what threshold it applied, when the tag moved.



The chain only exists if you build it, and a script is what proves it does:

ops/lineage.py: the audit answer, generated not narrated

Copy

```
from clearml import Model, Task

def lineage(model_id: str) -> dict:
    m = Model(model_id=model_id)
    t = Task.get_task(task_id=m.task)
    params = t.get_parameters()
    datasets = {k: v for k, v in params.items() if "dataset" in k.lower()}
    return {
        "model": {"id": m.id, "name": m.name, "tags": m.tags,
                  "published": m.published, "metadata": m.get_metadata()},
        "task": {"id": t.id, "name": t.name, "user": t.data.user,
                  "started": str(t.data.started), "status": t.status},
        "code": {"repo": t.data.script.repository,
                  "commit": t.data.script.version_num,
                  "entry": t.data.script.entry_point,
                  "dirty": bool(t.data.script.diff)},
        "data": datasets,
        "container": t.data.container,
    }
```



Code provenance	<code>task.data.script</code>	A dirty diff is recorded but not reviewed. May <code>dirty: true</code>
Data provenance	Parameters, via <code>alias</code>	A run reading a raw path records nothing
Approval trail	The promotion task's log and user	Tags moved by hand leave no trail
Immutability	<code>model.publish()</code>	An unpublished production model can be swapped silently
Retention of evidence	Your backups	A retention job that deleted the task breaks the chain

Make the promotion a task, never a click

A tag moved by hand in the UI has no reviewable record of what was compared or why. A promotion task has a user, a timestamp, a console log, the candidate and incumbent metrics, and the threshold it applied, which is what an auditor, or you at 2am, needs to read.

► TRY IT

1. Run the lineage function against your current production model and read the output.
2. Check whether `dirty` is true. If it is, the exact code is only reconstructable from the stored diff.
3. Check whether the `data` section names a dataset version or is empty.
4. Find the record of who promoted that model and what they compared. If it is a Slack message, that is the gap.

Expected result – usually a complete code chain, a missing data chain, and no approval record at all. Those two gaps are the difference between tracking experiments and being able to defend a deployed model.



addressed, immutable store

An erasure request under GDPR or a similar regime meets ClearML's design head-on. Datasets are immutable by intent, versions have children, and tasks reference them. This is the hardest problem in this track and it is primarily architectural, not operational.

What deletion has to touch:

Location	Contains	Deletable
Dataset chunks in your bucket	The raw records	Yes, but children break
Child dataset versions	Inherit parent chunks	Cascades
Task artifacts	Preprocessed copies, predictions	Yes, per task
Debug samples	Sample images, often the actual records	Yes, but easily forgotten
Model weights	Learned from the data; may be extractable	Effectively no
Elasticsearch	Console logs that may have printed records	Reindex or delete indices
Backups	Everything, historically	Only by expiring the backups

Weights are the part you cannot erase

Removing a subject's records from a dataset does not remove their influence from a trained model, and for some model classes information is extractable. If your regime treats a model as personal data, the only reliable remedy is retraining without those records, which is why the real fix is upstream: keep personal data out of the versioned store in the first place.



01 Version derived data, not raw personal data

Pseudonymise or aggregate before it enters a ClearML Dataset. The versioned artefact then holds no directly identifying records.

02 Keep the identity mapping outside ClearML

In a system built for deletion: a database with row-level deletes and an audit trail.

03 Crypto-shredding where raw data must be versioned

Encrypt per subject and destroy the key. The bytes remain, the content is unrecoverable, and the deletion is provable.

04 Never print records to the console

Console output goes to Elasticsearch and into your backups, and it is the most commonly forgotten copy.

05 Bound your backup retention deliberately

An indefinite backup is an indefinite copy. Erasure obligations and retention policy have to be reconciled on paper before they collide.

a deletion runbook that is at least honest about scope

Copy

```
from clearml import Dataset, Task

def find_references(dataset_id: str):
    """Every task whose parameters mention this dataset id."""
    hits = []
    for t in Task.get_tasks():
        if dataset_id in str(t.get_parameters()):
            hits.append((t.id, t.name, t.status))
    return hits

# 1. Enumerate the blast radius BEFORE deleting anything
refs = find_references("...dataset-id...")
print(f"{len(refs)} tasks reference this dataset")

# 2. Enumerate child dataset versions that inherit from it
```



► TRY IT

1. Pick one dataset and decide whether a subject could lawfully demand deletion of its contents.
2. If yes, run the reference finder and count the tasks and child versions affected.
3. List every other copy: task artifacts, debug samples, console logs, backups.
4. Write down what you would do, and how long it would take, if the request arrived tomorrow.

Expected result – the honest answer is usually "we could not do this cleanly", and knowing that before the request arrives is the entire value of the exercise. The architectural fix, versioning derived data only, is cheap now and expensive later.

Incident playbooks

Four incidents you should be able to work through without improvising.

The server is gone.

01 Agents keep running

Tasks in progress continue and buffer their reports. You have some time.

02 Stand up the server from the pinned image

Same version, not **latest**, because a different version means an unplanned migration on top of an incident.

03 Restore Mongo, then Elasticsearch

Mongo first: it is the system of record. Elastic second, for metrics and logs.

04 Verify in order



05

Reconcile

Tasks that finished during the outage may show as running. Mark them appropriately and note the gap.

Elasticsearch went read-only. Free disk (delete old indices or expand the volume), release the read-only block, then fix the cause: reporting rate, retention, or capacity. Symptom to recognise: metrics stop while training continues.

A model cannot be rebuilt. Run the lineage script; find the unpinned layer. It is nearly always the base image tag or an unpinned wheel. If the artifacts are gone, check whether a retention job deleted the task, then check bucket versioning. Record what you could not recover.

Someone enqueued something they should not have. Abort the task, revoke the credential that enqueued it, audit what that queue's agents could reach with their ambient credentials, and rotate anything exposed. Then fix the queue-level isolation that allowed it. Treat this as a credential compromise, not a mistake.

Incident	First action	The prevention
Server lost	Stand up the pinned image	Tested three-part backups; a known RTO
Elastic read-only	Free disk, release the block	Disk alerts; retention; reporting discipline
Model unrebuildable	Run lineage, find the unpinned layer	Image digests; lock files; weekly rebuild check
Unauthorised enqueue	Abort, revoke, rotate	Queue isolation; scoped credentials; no fork access
GPU queue full of nothing	List sessions, name the owners	Session timeouts; a weekly utilisation report
Storage bill spike	Break down by prefix and tag	Lifecycle rules; <code>save_top_k_tasks_only</code> ; retention

Write these down where the on-call person can find them



needs the first command, the verification step, and who to wake.

► TRY IT

1. Run the server-loss playbook on a scratch host, timing each step. That is your RTO.
2. Fill an Elasticsearch volume on a test instance and practise releasing the read-only block.
3. Run the rebuild check against a real production model and see whether it passes.
4. Write all four playbooks down and have someone who did not build the platform follow one.

Expected result – step four is the real test. A playbook that only works when its author runs it is not a playbook, and the gaps a second reader finds are exactly the ones that matter during an incident.

Where ClearML stops

Knowing the boundaries is a senior skill, and being able to name them is a strong interview signal.

Need	ClearML	Better tool
Point-in-time feature serving, online/offline parity	Not its job	A feature store: Feast, Tecton
SQL analytics over versioned tables	Datasets are file collections	Iceberg, Delta, or a warehouse
General-purpose workflow orchestration across a company	Pipelines are ML-shaped	Airflow, Dagster, Temporal
High-throughput, low-latency serving at scale	Serving exists, but is not a full mesh	KServe, Triton directly, a managed endpoint



Data quality and contract testing	No opinion	Great Expectations, dbt tests, Soda
Fine-grained RBAC and multi-workspace	Paid tier only	The paid tier, or a server per tenant
Petabyte-scale data versioning	<code>add_external_files</code> helps, but it is not a lakehouse	A table format with time travel
Model monitoring and drift as a product	Basic statistics via the serving stack	A dedicated monitoring platform

The honest summary

ClearML is strongest as the **experiment-to-model spine**: tracking, data versioning, remote execution, and a registry that a promotion workflow can drive. It is weakest as a general orchestrator, as a feature store, and as a serving mesh. A mature stack usually keeps ClearML for the spine and hands the boundaries to tools built for them, and says so out loud rather than stretching one tool over everything.

► TRY IT

1. For each row above, decide which side your team is on today.
2. Find one place where ClearML is being stretched past its shape. Name the cost of that.
3. Find one place where a second tool duplicates something ClearML already does well. Name the cost of that too.

Expected result – most stacks have one of each. Being able to argue both directions, where to adopt more ClearML and where to stop, is what distinguishes a senior answer from a preference.

Running ClearML as a platform



Publish the numbers

GPU utilisation, queue depth, storage by team, and the rebuild-check result. Visible without asking.



Ship a wrapper, not a wiki

Twenty lines that set the project prefix, output_uri, tags, and image. Conventions become defaults.



Own the upgrade cadence

A tested quarterly window with a rollback, rather than an emergency upgrade during someone's deadline.



Alert on the platform, not the users

Elastic disk, queue depth, services-agent liveness. Task failures belong to their authors.



Four runbooks, testable

Server lost, Elastic read-only, model unbuildable, unauthorised enqueue. Followed by someone else.

The monitoring set that matters, in priority order:

Alert	Threshold	Why first
Elasticsearch disk	> 80%	Read-only is silent and stops all metric reporting
Services agent down	No heartbeat for 10 min	Pipelines, schedulers, and triggers all stop; presents as "the nightly did not run"
Queue depth	Deep for > 1 hour	Dead worker, stuck session, or runaway sweep
MongoDB disk	> 80%	The system of record



API server 5xx rate	Any sustained rate	Agents fail to claim, users see hangs
Rebuild check	Failing	Reproducibility has silently regressed
GPU utilisation	< 40% weekly	You are paying for idle hardware

ops/platform_report.py: a weekly services task

Copy

```
from clearml import Task
from clearml.backend_api.session.client import APIClient

client = APIClient()
report = Task.current_task().get_logger()

for q in client.queues.get_all():
    depth = len(q.entries or [])
    report.report_single_value(f"queue_depth_{q.name}", depth)

workers = client.workers.get_all()
report.report_single_value("workers_online", len(workers))
report.report_single_value(
    "workers_idle",
    sum(1 for w in workers if not getattr(w, "task", None)),
)
```

The services agent is production, and it looks optional

Everything automatic (pipeline controllers, schedulers, dataset triggers, HPO optimisers, the serving control plane) lives on the services queue. When that one agent dies, nothing fails loudly: pipelines never start. It is the least glamorous and most load-bearing worker in the fleet, and it needs a liveness alert.

► TRY IT

1. Stop your services agent and see how long it takes anyone to notice. That gap is your detection time.
2. Build the seven alerts above, starting with Elasticsearch disk.
3. Publish one weekly report with utilisation, queue depth, storage by team, and the rebuild result.



because a pipeline that never starts produces no error anywhere. That is the single highest-value alert on the list.

The review checklist

Run this against any ClearML setup, yours or someone else's. It finds something every time.

Area	Check
Credentials	One credential per purpose? Nothing workspace-wide in automation? Rotation rehearsed?
Fork trust	Can a fork pull request enqueue anywhere? Do those agents hold cloud credentials?
Queue isolation	Are queues a security boundary, with different agent credentials per queue?
Diffs	Has anyone checked recent tasks' uncommitted changes for secrets?
TLS	Is the server behind TLS, with no plaintext credential path?
Storage	Are artifacts in object storage, not the file server? Lifecycle rules per prefix?
Backups	All three parts? Restored within the last quarter? RTO written down?
Version pinning	Server image tag pinned? SDK and agent versions paired across the team?
Elastic capacity	Disk alert at 80%? Retention job running? Reporting per epoch, not per batch?
Retention	Scheduled, exempting published models and keep tags, dry-run first?
Reproducibility	Base image by digest ? Lock file? Seeds connected? Weekly rebuild check green?
Lineage	Can you generate code, data, config, and approval for the production model in one command?



Promotion	A task with a threshold and a log, not a manual tag move? winner published?
Serving	Behind auth? Following a tag rather than an id? Canary path available?
Cost	GPU utilisation measured? Session timeouts? <code>save_top_k_tasks_only</code> on sweeps?
Services queue	Liveness alert? Treated as production?
Runbooks	Four written, and followed successfully by someone who did not write them?
Boundaries	Documented where ClearML stops and what owns the rest?

► TRY IT

1. Run the whole checklist against your own setup and count the failures.
2. Rank them by blast radius, not by effort. Fork trust and backups usually top the list.
3. Fix the top three this week and re-run.
4. Then run it against a colleague's project, which is where you learn how much of this is convention rather than enforcement.

Expected result – nobody passes on the first attempt. The two that matter disproportionately are an untested restore and a fork pull request with credentials. One is an unrecoverable data loss, the other is remote code execution.

The complete picture

Everything from all three levels in one platform. Read it as a whole; you should be able to justify every line and name which level it came from.

platform layout

Copy



```
| └─ train-pr.yaml          # rk gate. create, enqueue to ci, threshold
| └─ image.yaml            # builds the training image, tags by requirements hash
└─ requirements.lock       # hash-pinned
└─ Dockerfile.train       # CUDA + system libs + wheels; digest recorded on tasks
└─ src/{make_dataset,train,evaluate}.py
└─ platform/
  └─ clearml_wrapper.py    # project prefix, output_uri, tags, image – the governa
  └─ retention.py          # scheduled, dry-run-first, exempts published models
  └─ rebuild_check.py     # weekly rebuild of the production release
  └─ platform_report.py    # utilisation, queue depth, storage by team
  └─ lineage.py           # code + data + config + approval for a model id
  └─ promote.py           # threshold, tag move, publish – a task, not a click
└─ ops/{pipeline,hpo,triggers}.py
└─ runbooks/
  └─ server-lost.md
  └─ elastic-readonly.md
  └─ model-unrebuildable.md
  └─ unauthorised-enqueue.md
```

platform/clearml_wrapper.py: where every convention becomes a default

Copy

```
import os
from clearml import Task

IMAGE = "myregistry/train@sha256:9f2c4b19e0a7d3f1c6b8a2e4d7091f3b8a2e4d7091f3b8a2e4d70"

def init_task(project: str, name: str, queue: str | None = None, **kwargs) → Task:
    team = os.environ["ML_TEAM"]
    task = Task.init(
        project_name=f"{team}/{project}",
        task_name=name,
        output_uri=f"s3://ml-artifacts/{team}/clearml",    # never the file server
        reuse_last_task_id=False,                          # one task per run
        **kwargs,
    )
    task.add_tags([f"team:{team}", f"cost-centre:{os.environ['ML_COST_CENTRE']}"])
    task.set_packages("requirements.lock")                 # pinned, not detected
    task.set_base_docker(IMAGE, docker_arguments="--shm-size=8g") # by digest
    if queue:
        task.execute_remotely(queue_name=queue)
    return task
```



```
from platform.clearml_wrapper import init_task

task = init_task("experiments", "resnet18", queue="gpu")

import torch
from clearml import Dataset, OutputModel

opt = task.connect({"lr": 3e-4, "epochs": 20}, name="optimizer")
repro = task.connect({"seed": 42, "deterministic": True}, name="reproducibility")
data = task.connect({"dataset_version": "1.1.0"}, name="data")

set_seed(repro["seed"], deterministic=repro["deterministic"])
root = Dataset.get(dataset_project="datasets", dataset_name="iris",
                  dataset_version=data["dataset_version"],
                  alias="training data").get_local_copy()

logger = task.get_logger()
for epoch in range(opt["epochs"]):
    tr, va, acc = train_one_epoch(root, opt)
    logger.report_scalar("loss", "train", tr, epoch)      # per epoch, not per batch
    logger.report_scalar("accuracy", "val", acc, epoch)   # the HPO objective

test_acc = test(root)
logger.report_single_value("test_accuracy", test_acc)

model = OutputModel(task=task, name="resnet18-cls", framework="PyTorch")
model.update_weights(weights_filename="model.pt")
model.set_metadata("val_accuracy", test_acc, v_type="float")
model.set_metadata("dataset_version", data["dataset_version"])
model.tags = ["candidate"]                                # the gate promotes, not you
```

the fleet, as a security and cost design

Copy

```
# services – production. Everything automatic lives here. Liveness-alerted.
clearml-agent daemon --queue services --services-mode --cpu-only --detached

# cpu / gpu – researcher work. Data read + artifact write, no delete.
clearml-agent daemon --queue cpu --cpu-only --detached
clearml-agent daemon --queue gpu --gpus 0 --docker $IMAGE --detached
clearml-agent daemon --queue gpu --gpus 1 --docker $IMAGE --detached

# ci – protected branches only. Artifact write, nothing else.
```



```
clearml-agent daemon --queue ci-untrusted --docker $IMAGE --detached
```

the scheduled work that makes it a platform

Copy

```
# On the services queue, all of them tasks with logs:
python platform/retention.py          # daily, dry-run gated
python platform/rebuild_check.py      # weekly – reproducibility as a monitored property
python platform/platform_report.py    # weekly – utilisation, depth, storage by team
python ops/triggers.py                # dataset version → pipeline
```

Eighteen decisions in there span the whole series:

Decision	Level
<code>Task.init</code> before framework imports	Beginner
Read parameters from <code>connect</code> 's return value	Beginner
One <code>title</code> , several <code>series</code> ; single values for finals	Beginner
<code>output_uri</code> to object storage, never the file server	Beginner
Dataset consumed with <code>alias</code> , version as a parameter	Beginner
<code>execute_remotely</code> as the local/remote switch	Beginner
<code>reuse_last_task_id=False</code> on every experiment	Mid
Packages declared, not detected	Mid
Controllers on <code>services</code> , never on GPU	Mid
HPO objective verified against the base task	Mid
Pipeline caching made correct by versioning inputs	Mid
Promotion by tag, so serving self-deploys	Mid



Queues as a security boundary, with per-queue credentials	Senior
Fork pull requests on ephemeral, credential-free agents	Senior
Base image pinned by digest , plus a lock file	Senior
Three-part backups, restored and timed	Senior
A weekly rebuild check, so reproducibility is monitored	Senior
A wrapper that makes every convention a default	Senior

► TRY IT: THE ONE THAT MATTERS

1. Write the wrapper for your own team and migrate one project onto it. Measure how much researcher-facing code disappeared.
2. Split your queues by trust level and give each pool its own credentials. Verify a **ci-untrusted** task can reach nothing.
3. Take all three backups and restore them onto a scratch host. Publish the measured RTO.
4. Schedule the rebuild check and let it fail. Fix the unpinned layer it finds, then let it run green for a month.
5. Run the review checklist and publish the results, including what you are not going to fix and why.

Expected result – step five is the senior deliverable. A checklist with honest, argued exceptions is worth more than a green one, because it shows the trade-offs were made deliberately rather than by omission, and it gives whoever inherits the platform a map instead of a mystery.

Where the series leaves you

Across three levels you have gone from two lines in a script to owning a platform. You can track an experiment automatically and log deliberately; version data incrementally and consume it reproducibly; clone and re-launch any run on remote hardware; run HPO with a



model, tested backups, monitored reproducibility, an audit chain, and runbooks somebody else can follow.

The question a senior gets asked	The answer this series gives
"Can you reproduce the model we shipped in March?"	Clone its task; the digest, lock file, seed, and dataset version are all recorded, and the weekly rebuild check already proved it
"Who can run code on our GPUs?"	Whoever can enqueue: the queues, their credentials, and the fork policy
"What does this platform cost?"	GPU-hours, artifact and dataset storage, server infrastructure, and egress: broken down by team, published weekly
"What breaks first if the team triples?"	Elasticsearch on reporting rate, then MongoDB on task count, with a retention and capacity plan to match
"If the server dies, how long until we are back?"	A measured RTO from a restore we ran last quarter
"Prove this model is what we think it is"	One command producing code, data, config, and the timestamped approval
"Should we use ClearML for X?"	Where it is strongest, where it stops, and what owns the rest

The through-line of all three levels is a single idea: **a recorded run that can be executed again is worth more than a log of one that cannot.** Everything else (the agents, the queues, the registry, the pinning, the backups) exists to keep that property true as the number of runs, people, and months grows.



PREVIOUS

Mid-level · Tips & Tricks

NEXT UP

Senior · Interview Review





MLOps MENA
COMMUNITY

العربية العربية



[Join the Community ↗](#)



MLOps MENA
COMMUNITY

Free MLOps and AI learning for engineers across the Middle East and North Africa — live sessions, three structured roadmaps, open cohort courses, study groups, and mentorship. We also train, staff, and build for companies across the region.



LEARN

[Roadmaps](#)

[Student Guides](#)

[Courses](#)

[The MLOps Practitioner](#)

[Sessions](#)

[Articles](#)

COMMUNITY

[Meet the team](#)

[Services for companies](#)

[FAQ](#)

[Contact us](#)

[Privacy Policy](#)

WHERE WE ARE

[WhatsApp](#)

[Discord](#)

[LinkedIn](#)

[X](#)

[YouTube](#)

[GitHub](#)



MLOps MENA
COMMUNITY

العربية



Join the Community ↗



Learning space

Free to join this week with code **FREECORETEAM**

© 2026 MLOps MENA Community. Free learning for the community.

Built by the community, for the community.